



Кафедра 806

Программа Машинное обучение и анализ данных

Квалификация: инженер-исследователь

На тему: Разработка интеллектуального модуля интерактивного анализа данных в реляционных базах данных

Рецензент: _____ TODO: ФИО 3 (_____) _____

мая 2025 года

Москва 2025



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Обучающийся _____ Гимазетдинов Дмитрий Русланович
Институт (Филиал) _____ Институт №8 Кафедра _____ 806
Группа М8О-20СВ-24 Направление подготовки 02.04.02 Фундаментальная информатика
и информационные технологии
Программа _____ Машинное обучение и анализ данных
Квалификация: инженер-исследователь
Наименование темы: Разработка интеллектуального модуля интерактивного анализа
данных в реляционных базах данных
Руководитель: Судаков Владимир Анатольевич

Выпускная квалификационная работа студента Иванова Ивана Ивановича на
тему «Разработка информационной системы учёта заявок» выполнена на
высоком уровне.

Студент проявил инициативу, самостоятельность и ответственность при
выполнении всех этапов работы. Материал изложен логично, последовательно
и грамотно.

Полученные результаты имеют практическую ценность и могут быть
использованы для дальнейшего совершенствования систем автоматизации.

Работа соответствует предъявляемым требованиям к ВКР, а её автор
заслуживает оценки «отлично» и допуска к защите.

Работа проверена на объем заимствования. % заимствования -

Заключение: _____ Все чики пуки

_____ 2025 г.

Руководитель _____



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Обучающийся _____ Гимазетдинов Дмитрий Русланович

Институт (Филиал) _____ Институт №8 _____ Кафедра _____ 806

Группа М8О-20СВ-24 Направление подготовки 02.04.02 Фундаментальная информатика
и информационные технологии _____

Программа _____ Машинное обучение и анализ данных _____

Квалификация: инженер-исследователь _____

Наименование темы: Разработка интеллектуального модуля интерактивного анализа
данных в реляционных базах данных _____

Руководитель: Судаков Владимир Анатольевич _____

Выпускная квалификационная работа студента Иванова Ивана Ивановича на
тему «Разработка информационной системы учёта заявок» выполнена на
высоком уровне.

Студент проявил инициативу, самостоятельность и ответственность при
выполнении всех этапов работы. Материал изложен логично, последовательно
и грамотно.

Полученные результаты имеют практическую ценность и могут быть
использованы для дальнейшего совершенствования систем автоматизации.

Работа соответствует предъявляемым требованиям к ВКР, а её автор
заслуживает оценки «отлично» и допуска к защите.

Заключение: _____ Все чики пуки _____

_____ 2025 г.

Консультант _____

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	3
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	4
ВВЕДЕНИЕ	5
0.1 АКТУАЛЬНОСТЬ ТЕМЫ	5
0.2 НАУЧНАЯ НОВИЗНА	6
ТЕОРИЯ	9
0.3 Концепция интерфейсов доступа к данным на естественном языке (NLIDB)	10
0.3.1 Краткая история NLIDB	11
0.4 Задача Tex to SQL	14
0.4.1 Формулировка задачи text-to-SQL	15
0.4.2 Архитектурные подходы и современные методы Text-to-SQL	15
0.4.3 Оценка и Бенчмарки	18
0.5 Технология RAG в задачах взаимодействия с базами данных . .	20
0.5.1 Определение и сущность Retrieval-Augmented Generation (RAG)	20
0.5.2 Эволюция и разновидности архитектур RAG	21
0.5.3 Проблема интеграции полного контекста базы данных в Text-to-SQL	22
0.5.4 Механизм RAG для извлечения релевантного контекста схемы БД	23
0.6 Методология комплексной оценки систем Text-to-SQL	24
0.6.1 Формальные метрики соответствия кода	24
0.6.2 Продвинутые метрики и бенчмарки (BIRD и Spider)	25
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	28

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей выпускной квалификационной работе магистра применяют следующие термины с соответствующими определениями:

L^AT_EX — система компьютерной вёрстки

МАИ — Московский Авиационный Институт

S^2 — формула в списке обозначений

SQL — (subquery language) - язык запросов

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей выпускной квалификационной работе магистра применяют следующие сокращения и обозначения:

ИИ — Искусственный интеллект

LLM — Large Language Model

КХД — Корпоративное хранилище данных

РБД — Реляционные базы данных

NLP — Natural Language Processing

RAG — Retrieval-Augmented Generation

ВВЕДЕНИЕ

В современных условиях цифровой трансформации все больше сфер сталкивается с существенным ростом объема данных и усложнением процессов их обработки. Обеспечение надежного и своевременного формирования отчетности для регулирующих органов, включая Центральный банк, заказчики и д.р., требует использования эффективных и масштабируемых решений.

КХД играют ключевую роль в поддержке аналитики и принятия решений, предоставляя централизованный доступ к структурированной информации. Однако эксплуатация и сопровождение таких систем сопряжены с высокими трудозатратами, необходимостью обеспечения высокой производительности и надежности, а также сложностями в обработке ошибок и оптимизации запросов.

С развитием технологий ИИ и появлением больших языковых моделей LLM открываются новые возможности для автоматизации процессов сопровождения КХД. ИИ-агенты, основанные на LLM, способны эффективно решать задачи: *прогнозирования сбоев, выявления аномалий, автоматической генерации аналитических отчетов и предоставления рекомендаций по оптимизации*. Интеграция таких интеллектуальных компонентов в архитектуру КХД позволяет снизить нагрузку на технический персонал, повысить качество и скорость анализа данных, а также улучшить качество обслуживания регуляторных требований.

0.1 АКТУАЛЬНОСТЬ ТЕМЫ

В последние годы наблюдается стремительный рост объёмов данных и активное внедрение РБД в различные сферы экономики, науки и управления. Однако стандартные средства взаимодействия с РБД через язык SQL остаются труднодоступными для пользователей, не обладающих навыками программирования. Решение задачи формирования и выполнения SQL-запросов на основе естественно-языковых запросов пользователя позволит значительно повысить доступность аналитических инструментов и эффективность работы с данными.

Особую актуальность данной работы определяет развитие технологий искусственного интеллекта, появление новых подходов в области обработки

естественного языка (NLP) и генерации SQL-запросов с использованием больших языковых моделей (LLM). Интеллектуальные чат-агенты, интегрированные с РБД, позволяют не только автоматизировать формирование корректных SQL-запросов, но и обеспечивают наглядную визуализацию результатов анализа, что повышает удобство и скорость принятия решений.

Современные архитектурные решения предусматривают использование таких инструментов, как Python-фреймворки FastAPI и Streamlit, современные СУБД (например, PostgreSQL) и LLM-фреймворки (LangChain, Ollama) для построения полноценных интеллектуальных систем. Разработка чат-агента с поддержкой RAG технологий предоставляет возможность формирования релевантного контекста для сложных пользовательских запросов, расширяя возможности аналитики и делая её по-настоящему интерактивной.

Таким образом, исследование и реализация интеллектуального чат-агента для работы с реляционными базами данных на естественном языке представляет собой не только научный, но и практический интерес, обеспечивая эффективную обработку и анализ больших объёмов данных в реальном времени.

0.2 НАУЧНАЯ НОВИЗНА

В банковской сфере наблюдается устойчивый тренд перехода от использования типовых готовых программных продуктов к разработке собственных интеллектуальных систем сопровождения корпоративных хранилищ данных (КХД). Так, например, ПАО «Промсвязьбанк» (ПСБ) инициировал сотрудничество с профильными образовательными учреждениями, в частности с Московским авиационным институтом, где была создана продуктовая модель для анализа кредитоспособности клиентов. Это демонстрирует растущую потребность банков в кастомизированных, гибких и интеллектуальных решениях, способных адаптироваться к специфике внутренних бизнес-процессов и требованиям регуляторов.

Современные инструменты интеграции и сопровождения данных, такие как Oracle Data Integrator и Oracle GoldenGate, широко применяются для решения задач интеграции, репликации и управления потоками данных. Несмотря на широкое применение Oracle Data Integrator и GoldenGate для интеграции и репликации данных, текущие процессы зачастую требуют

ручной настройки «запинок» — специалист формирует, из каких систем и таблиц извлекать данные и как организовать их трансформацию. Это ограничивает скорость адаптации и гибкость систем, требует высокой квалификации и значительных временных затрат.

Традиционные алгоритмы оптимизации SQL-запросов в этих решениях не всегда справляются с динамически меняющимися сценариями, что негативно влияет на производительность.

Интеллектуальные агенты на базе больших языковых моделей способны значительно расширить возможности сопровождения КХД. Они могут:

- самостоятельно анализировать структуру и содержимое данных;
- автоматически выявлять релевантные таблицы и источники информации;
- формировать и оптимизировать SQL-запросы с учётом текущей нагрузки и особенностей базы данных;
- обеспечивать полноценный цикл сопровождения с помощью методов обратного проектирования (reverse engineering), минимизируя ручное вмешательство и снижая риски ошибок.

Такой подход открывает новые горизонты для повышения эффективности, устойчивости и автоматизации корпоративных хранилищ данных в банковской сфере.

Данный подход требует высокой квалификации, занимает значительное время и ограничивает гибкость и адаптивность систем. Кроме того, традиционные алгоритмы оптимизации SQL-запросов, применяемые в этих инструментах, не всегда способны эффективно справляться с динамически меняющимися, комплексными сценариями обработки данных, что сказывается на производительности и времени отклика.

Научная новизна данной работы заключается в применении больших языковых моделей (LLM) для построения интеллектуальной системы сопровождения КХД, которая обеспечивает:

- автоматизированный и адаптивный анализ SQL-запросов и процедур с динамическим обучением и самосовершенствованием алгоритмов, направленный на повышение производительности и снижение нагрузки на инфраструктуру;
- интеллектуальное выявление аномалий и диагностику ошибок на основе комплексного анализа логов, метрик и структурированных

данных с генерацией понятных и обоснованных рекомендаций на естественном языке, что облегчает взаимодействие с техническими специалистами;

- автоматическую генерацию регламентированных отчетов, полностью соответствующих требованиям банковского регулирования, с возможностью оперативного обновления моделей в соответствии с изменениями нормативной базы;
- интеграцию ИИ-агентов в процессы мониторинга и сопровождения КХД, обеспечивающую проактивный контроль, предиктивную аналитику и значительное сокращение трудозатрат на сопровождение.

Особенно перспективным является использование методов обратного проектирования (reverse engineering), благодаря которым ИИ-агенты способны полностью автоматизировать цикл сопровождения — от анализа архитектуры данных и оптимизации процессов интеграции до формирования отчетности. Это выводит разработанную систему за рамки традиционных инструментов, создавая комплексный, интеллектуальный и адаптивный продукт, отвечающий современным вызовам банковского сектора.

ТЕОРИЯ

Корпоративные хранилища данных (КХД) являются критически важными компонентами банковских информационных систем, обеспечивая централизованное хранение, интеграцию и аналитическую обработку больших объемов данных. КХД поддерживают процессы формирования отчетности для регуляторов, включая Центральный банк, а также служат основой для принятия управленческих и стратегических решений. Несмотря на свою важность, эксплуатация КХД сталкивается с рядом системных проблем, существенно влияющих на качество и эффективность работы банковских ИТ-инфраструктур.

Во-первых, одна из ключевых проблем — сложность и неоптимальность обработки запросов. Высокая нагрузка на ресурсы серверов баз данных возникает из-за плохо оптимизированных SQL-запросов и хранимых процедур, что приводит к увеличению времени отклика и снижению общей производительности системы. Учитывая рост объемов данных и сложность аналитических задач, классические методы оптимизации зачастую оказываются недостаточно эффективными, требуя постоянного ручного вмешательства и экспертизы со стороны специалистов.

Во-вторых, важным вызовом является оперативная диагностика и устранение аномалий. Текущие системы мониторинга, как правило, ориентированы на реактивный подход: проблемы выявляются только после появления сбоев, нарушений целостности данных или деградации производительности. Это приводит к рискам прерывания бизнес-процессов, снижению качества обслуживания клиентов и нарушению регуляторных требований. Отсутствие проактивного контроля и прогнозирования затрудняет своевременное выявление потенциальных проблем.

В-третьих, значительные затраты времени и ресурсов связаны с ручным сопровождением и анализом состояния КХД. Технические специалисты вынуждены вручную обрабатывать логи, интерпретировать сообщения об ошибках и формировать отчеты, что снижает оперативность реагирования и увеличивает вероятность ошибок. Кроме того, традиционные инструменты мониторинга и поддержки не всегда способны адаптироваться к динамично меняющимся условиям эксплуатации, что ограничивает масштабируемость и устойчивость систем.

Наконец, возрастание требований к скорости и качеству аналитики, а также усложнение регуляторных стандартов, предъявляют новые вызовы к сопровождению КХД. Необходимость своевременного формирования точной отчетности и обеспечения высокой доступности данных требует внедрения инновационных подходов, способных автоматизировать рутинные процессы и повысить надежность систем.

В совокупности перечисленные проблемы обосновывают актуальность внедрения современных технологий искусственного интеллекта и больших языковых моделей, которые способны реализовать интеллектуальный мониторинг, автоматизированное обнаружение и коррекцию ошибок, а также генерацию аналитических отчетов с минимальным участием человека, что существенно повысит эффективность сопровождения КХД в банковском секторе.

0.3 Концепция интерфейсов доступа к данным на естественном языке (NLIDB)

Интерфейс естественного языка к базам данных (NLIDB) — это система, разработанная для предоставления пользователям, не обладающим техническими знаниями или опытом работы с языками запросов, такими как SQL, возможности запрашивать информацию из баз данных, используя естественный язык [history and usability].

- а) **Назначение:** NLIDB устраняет барьер взаимодействия между пользователями и базами данных, позволяя обращаться к информации интуитивно и эффективно. [natural language.pdf]
- б) **Основная функция:** Задача NLIDB заключается в преобразовании запроса на естественном языке (NLQ) в исполняемый язык базы данных (например, SQL) [review.pdf]. В контексте реляционных баз данных это также известно как задача семантического парсинга "текст в SQL" (Text-to-SQL semantic parsing task) [нейросетевые подходы.pdf].
- в) **Целевая аудитория:** Система нацелена на пользователей без технической экспертизы, которым трудно формулировать запросы на специализированных языках, таких как SQL [history and usability].

0.3.1 Краткая история NLIDB

Историческая траектория развития интерфейсов естественного языка к базам данных (NLIDB) представляет собой увлекательный путь от жестко детерминированных систем, основанных на лингвистических правилах, до современных вероятностных моделей, использующих глубокое обучение. Анализ этого перехода позволяет выявить фундаментальные причины, по которым NLIDB только сейчас, в эпоху больших языковых моделей (LLM), приближаются к широкой коммерческой применимости.

I. Эра детерминизма и ограниченного домена (1960-е - 1990-е годы)

Ранние системы NLIDB, появившиеся в 1960-х годах, такие как BASEBALL и LUNAR, строились на детерминированном (символьном) подходе. Основная идея заключалась в кодировании языка с помощью заранее определенных правил и грамматик.

Ключевые особенности и ограничения этого периода:

- а) **Доменно-зависимый дизайн (Domain Dependence):** Большинство этих систем разрабатывались с учетом конкретной базы данных или ограниченного домена знаний. Внедрение изменений или перенос на другую предметную область требовал значительных ручных усилий по адаптации или полному переписыванию правил. [history and usability.pdf]
- б) **Техники перевода:** Использовались методы сопоставления с образцом (pattern matching) и, позднее, семантические грамматики (semantic grammar). Семантические грамматики предлагали более богатую семантическую информацию, но при этом жестко связывали знания о предметной области с самой грамматикой (hard-wired into the semantic grammar). [history and usability.pdf]
- в) **Проблемы производительности:** Несмотря на разработку множества прототипов, в том числе коммерчески доступных в 1980-х (например, INTELLECT), их производительность не была удовлетворительной. Системы испытывали трудности с обработкой неоднозначности естественного языка (language ambiguity), семантического эллипсиса и не могли четко объяснить пользователю границы своих возможностей (language coverage). [history and usability.pdf]

II. Переход к статистическим и гибридным подходам (2000-е - 2010-е годы)

На рубеже веков исследования перешли к более эмпирическим (статистическим) и гибридным методам, которые опирались на анализ текстовых корпусов.

- а) Эмпирические подходы: Они использовали статистический анализ для автоматического получения и изучения более сложных знаний, хотя их эффективность сильно зависела от качества и размера обучающего корпуса. [history and usability.pdf]
- б) Гибридные методы: Комбинирование правил с машинными методами (например, TypeSQL, NALMO) позволяло использовать правила для задач с явной логикой и машинное обучение для более сложных семантических проблем. [history and usability.pdf]
- в) Нейросетевая революция (Seq2Seq): Появление больших кросс-доменных датасетов (WikiSQL, Spider) и успехи архитектуры sequence-to-sequence (Seq2Seq) в машинном переводе привели к возрождению интереса к NLIDB. Задача Text-to-SQL начала рассматриваться как задача трансдукции последовательности. Нейросетевые модели, такие как SyntaxSQLNet и IRNet, сфокусировались на улучшении кодирования запроса и схемы (schema encoding) и введении грамматически осведомленных декодеров для повышения синтаксической корректности SQL. [history and usability.pdf]

III. Эпоха LLM: Реальность вероятностного и генеративного подхода

В настоящее время NLIDB переживают качественный прорыв благодаря появлению больших языковых моделей (LLM), основанных на архитектуре Transformer [review.pdf]. LLM представляют собой вершину вероятностного и генеративного подхода.

Неудовлетворительная работа ранних систем была обусловлена жесткостью детерминированных (символьных) правил. Эти системы не могли масштабироваться, обрабатывать естественную неоднозначность и изменчивость языка, а также переносить знания между предметными областями. Они требовали, чтобы пользователь формулировал запрос в пределах строго ограниченного языкового покрытия. [review.pdf]

Успех LLM обусловлен следующими факторами:

Таблица 1 – Сравнение детерминированного и генеративного подходов к созданию NLIDB

Характеристика	Ранние NLIDB	Генеративные (LLM)
Основной принцип	Строгое соответствие предопределенным правилам и грамматикам	Статистическое моделирование контекста и семантики на основе огромных корпусов
Обработка неоднозначности	Сбой или запрос уточнений (если предусмотрен диалог)	Использование контекста и обширных знаний для вероятностного разрешения неоднозначности
Переносимость (Domain Independence)	Низкая. Требуется ручная настройка грамматик и словарей	Высокая. Могут обобщать и применять знания в кросс-доменных сценариях
Результат	SQL-запрос, если найдено точное соответствие правилу.	Итеративная генерация и уточнение SQL, а также генерация ответа на естественном языке

- а) **Глубокое контекстуальное и семантическое понимание:** LLM, обученные на обширных массивах данных, способны улавливать глубокую структуру и семантическую информацию естественного языка. Это позволяет им справляться с лингвистическими явлениями, такими как эллипсис и анафора, которые были серьезными проблемами для ранних систем.
- б) **Интеграция знаний о предметной области:** Современные методы, основанные на генеративном ИИ, используют векторные базы данных для хранения и извлечения релевантных фрагментов структурных метаданных и сложных бизнес-правил, что критически важно для обеспечения семантической корректности генерируемого SQL.
- в) **Автоматическая валидация и уточнение:** LLM используются не только для генерации SQL, но и для двухэтапной автоматической проверки: синтаксической и семантической (интроспекции). Если обнаружены ошибки, система автоматически и итеративно уточняет запрос, что значительно повышает надежность и точность, необходимую для бизнес-приложений.
- г) **Удобство для пользователя:** В отличие от предыдущих систем, которые часто возвращали только необработанный результат SQL, современные LLM генерируют хорошо сформулированные, контекстуально соответствующие ответы на естественном языке.

Это устраняет необходимость в технической экспертизе SQL со стороны конечного пользователя, демократизируя доступ к данным.

Таким образом, переход от жестких, доменно-зависимых систем, ограниченных ручными правилами, к гибким, масштабируемым, основанным на трансформерах моделям, способным к глубокому контекстуальному вероятностному рассуждению, является ключевым фактором, сделавшим NLIDB реальностью для широкого использования.

0.4 Задача Text to SQL

Text-to-SQL — это задача, которая заключается в автоматическом переводе вопросов или запросов на естественном языке (NL) в запросы на языке структурированных запросов (SQL), которые могут быть выполнены на заданной реляционной базе данных для получения ответа

В задаче Text to SQL имеется несколько исторически сложившихся аспектов:

- а) **Преобразование языка:** Text-to-SQL функционирует как задача преобразования последовательности в последовательность (sequence-to-sequence problem);
- б) **Входные данные:** На вход подаётся вопрос на естественном языке (Q) и схема базы данных (D или S), которая включает информацию о таблицах, столбцах, их типах данных, первичных и внешних ключах, а также другие метаданные;
- в) **Цель:** сгенерировать точный и допустимый SQL-запрос ($\hat{Y}$), который корректно выражает намерение пользователя и может быть выполнен для получения требуемого результата из базы данных;
- г) **Упрощение доступа:** Системы Text-to-SQL значительно упрощают доступ к данным для пользователей, не обладающих техническими навыками или специализированными знаниями SQL, позволяя им взаимодействовать с базами данных с помощью обычного языка

Подходы к решению данной задачи постоянно менялись, начиная от ранних методов, таких как **rule-based methods**, которые позже заменили подходы основанные на глубоком обучении, включая архитектуры типа «кодировщик-декодировщик» (encoder-decoder), такие как **LSTM** и **трансформеры**.

В последнее время доминирующим направлением стало использование

больших языковых моделей (LLM), таких как серии GPT, благодаря их продвинутым возможностям генерации текста и рассуждений. Поэтому в контексте LLM основная проблема Text-to-SQL сводится к разработке оптимального промпта (prompt engineering), чтобы модель сгенерировала правильный SQL-запрос.

0.4.1 Формулировка задачи text-to-SQL

Задача: Обработка вопроса на **NL** в запрос **SQL**.

Входные и выходные данные:

Таблица 2 – Компоненты системы Text-to-SQL

Компонент	Input	Output	Форма
Основной запрос	Вопрос на ест. языке (Q)	SQL-запрос ($\hat{Y}$)	Прямая генерация
Схема БД	Структура БД (таблицы, столбцы, ключи)	—	—
Промпт	Инструкция задачи	—	Цепочка рассуждений (CoT)
Контекст	Примеры Q/SQL (few-shot)	—	Самокоррекция
Дополнения	Правила/ограничения	—	CTEs (структурир. SQL)

0.4.2 Архитектурные подходы и современные методы Text-to-SQL

В современных системах Text-to-SQL, основанных на LLM, используется несколько ключевых архитектурных подходов, которые часто комбинируются для повышения точности и эффективности

А. Промпт-инжиниринг (Prompt Engineering)

Это основной подход, который лежит в основе LLM-based Text-to-SQL и не требует дополнительного обучения модели. Он включает в себя:

- а) Представление вопроса (Question Representation) Это то, как схема базы данных и вопрос представляются LLM. Рекомендуемые форматы:
 - Code Representation Prompt (CR): Представление структурированных знаний с использованием синтаксиса SQL (например, операторов CREATE TABLE), что включает полную информацию о базе данных, включая

первичные и внешние ключи (Foreign Keys, FK). Этот подход часто оказывается предпочтительным, поскольку стимулирует способность LLM к работе с кодом

- OpenAI Demonstration Prompt (OD): Использование специальных символов (например, #) для комментирования схемы и добавление конкретных инструкций, например, «Complete sqlite SQL query only and with no explanation» (что доказало свою эффективность)

б) Обучение в контексте (In-Context Learning / Few-Shot) Предоставление модели нескольких примеров для изучения шаблонов преобразования. Ключевые аспекты этого метода:

- Выбор примеров (Example Selection): Важно выбирать наиболее полезные примеры (Q). Методология DAIL Selection (DAIL) показала высокую эффективность, поскольку она учитывает сходство как самого вопроса, так и предполагаемого SQL-запроса (скелета запроса), что важно для обучения LLM сопоставлению вопроса с запросом
- Организация примеров (Example Organization): Для баланса между качеством информации и количеством примеров (что ограничено лимитом токенов) была предложена DAIL Organization (DAIL). Этот метод представляет пары «вопрос-SQL» без повторного включения полной схемы базы данных в каждый пример, что повышает эффективность использования токенов

в) Усовершенствование рассуждений (Reasoning Enhancement) Эти методы направлены на повышение способности LLM обрабатывать сложные запросы

- Chain of Thought (CoT): Побуждение модели к генерации промежуточных шагов рассуждения (например, с помощью фразы "Let's think step by step") для разбиения сложной задачи на более простые, управляемые этапы
- Query Decomposition (QDecomp) / Least-to-Most: Разделение сложного NL-запроса на последовательность

логических шагов или подзапросов

- ACT-SQL (Automatic CoT): Метод, автоматически создающий примеры CoT для обучения моделей, что делает процесс более масштабируемым
- Self-Consistency: Генерация нескольких потенциальных SQL-запросов и выбор наиболее согласованного из них (например, с использованием механизма голосования), что повышает надёжность

В. Тонкая настройка (Supervised Fine-Tuning, SFT)

Тонкая настройка LLM (или малых языковых моделей, SLM) на специализированных датасетах Text-to-SQL (например, Spider, BIRD) — это альтернативный или дополнительный подход, который значительно повышает производительность, особенно для открытых моделей

- Для открытых LLM/SLM:** Тонкая настройка необходима для повышения точности и конкурентоспособности. Например, модель SQL_DS_100M_ft, основанная на открытой архитектуре CodeS, после контролируемой тонкой настройки достигла высокой точности (до 72.2% EX на тестах BIRD) при низких вычислительных затратах.
- Типы настройки:** Используется как полная тонкая настройка (Full-Parameter Fine-Tuning) для достижения максимальной точности в специфических задачах, так и параметрически эффективная тонкая настройка (Parameter-Efficient Fine-Tuning), которая экономит ресурсы, настраивая только часть параметров.
- Ограничение:** После тонкой настройки LLM могут терять способность к обучению в контексте (in-context learning).

С. Архитектуры ИИ-Агентов (LLM Agent Methods)

Рассмотрим современные многоэтапные архитектуры, которые сочетают промпт-инжиниринг с механизмами самокоррекции и внешними инструментами:

- DIN-SQL:** Эта архитектура декомпозирует задачу Text-to-SQL на четыре управляемых этапа:
 - связывание схемы (Schema Linking);
 - классификация и декомпозиция запроса (например, на простой, не вложенный, вложенный)

– генерация SQL и самокоррекция (Self-Correction).

Для сложных запросов (nested complex queries) используется разбиение проблемы на несколько шагов, при котором сначала решаются подзапросы, а затем генерируется окончательный ответ

- б) DAIL-SQL: Хотя это в первую очередь метод промпт-инжиниринга, он является интегрированным решением, которое использует эффективное представление кода (CR), интеллектуальный выбор примеров (DAIL) и эффективную организацию (DAIL). DAIL-SQL может быть дополнительно оснащён механизмом самосогласованности (Self-Consistency) для повышения точности.
- в) MAC-SQL: Это фреймворк для совместной работы нескольких агентов, который использует Decomposer (для пошагового разбиения сложных запросов), Selector (для ограничения базы данных релевантными данными) и Refiner (для исправления ошибочных SQL-запросов с использованием внешних инструментов выполнения SQL)
- г) SQL-CRAFT: Интегрирует интерактивные циклы исправления (Interactive Correction Loop, IC-Loop) на основе обратной связи от системы управления базами данных (СУБД) и Python-усиленное рассуждение (Python-enhanced reasoning) для логических операций и расчётов перед генерацией SQL

0.4.3 Оценка и Бенчмарки

Оценка и бенчмаркинг в области Text-to-SQL с использованием больших языковых моделей (LLM) фокусируются на нескольких ключевых метриках, которые оценивают точность, корректность выполнения и эффективность сгенерированных SQL-запросов.

Основные показатели

- а) Точность выполнения (Execution Accuracy, EX)
- б) Точность точного совпадения (Exact Matching Accuracy, EM)
- в) Эффективность использования токенов (Token Efficiency / Cost Efficiency)
- г) Показатель достоверной эффективности (Valid Efficiency Score, VES)
- д) Точность тестового набора (Test-suite Accuracy, TS)

Точность выполнения (EX) является основным и наиболее важным

показателем для оценки эффективности LLM-based решений. EX сравнивает результат выполнения (output) сгенерированного SQL-запроса с результатом выполнения эталонного (ground truth) SQL-запроса на экземплярах базы данных.

Точность точного совпадения (EM) — это более строгая метрика, которая измеряет синтаксическую корректность сгенерированного запроса. EM требует, чтобы сгенерированный SQL-запрос в точности совпадал (включая ключевые слова) с эталонным SQL-запросом. Но из-за разнообразия синтаксиса, эталонный SQL-запрос может быть не единственным корректным вариантом.

Эффективность использования токенов (Token Efficiency / Cost Efficiency) Эта метрика не является мерой корректности, но является критически важной для практического применения Text-to-SQL, особенно при использовании проприетарных LLM, таких как модели OpenAI, которые взимают плату за токены.

Показатель достоверной эффективности (Valid Efficiency Score, VES) — это комплексный показатель, который оценивает как корректность, так и эффективность выполнения сгенерированного запроса. Он учитывает достоверность запроса (успешность выполнения и соответствие результата эталону) и эффективность запроса (скорость выполнения по сравнению с эталонным запросом).

Точность тестового набора (Test-suite Accuracy, TS) - эта метрика разработана для оценки производительности моделей на разнообразном и сфокусированном наборе тестовых баз данных. TS измеряет, насколько хорошо модель предсказывает SQL-запросы, которые дают правильные результаты в различных сценариях баз данных, и стремится измерить строгий верхний предел семантической точности

Бенчмарки

Оценка LLM-based Text-to-SQL решений проводится на основе крупномасштабных кросс-доменных наборов данных, которые служат стандартами для сравнения:

- а) Spider: Это крупномасштабный кросс-доменный набор данных, который является ключевым для Text-to-SQL. Он содержит сложные SQL-запросы. Производительность на нем измеряется, главным образом, Точностью выполнения (EX), и на его лидерборде

ведущие решения основаны на LLM.

- б) BIRD: Бенчмарк, ориентированный на крупномасштабные базы данных, который стремится устранить разрыв между академическими исследованиями и реальными приложениями. BIRD специально исследует проблемы, связанные с большими базами данных с «грязными» значениями, необходимостью вывода внешнего знания и оптимизацией эффективности выполнения SQL.
- в) Spider-Realistic: Более сложный вариант Spider, где вопросы были вручную переработаны для моделирования более реалистичных сценариев, при этом эталонные SQL-запросы остались неизменными.

0.5 Технология RAG в задачах взаимодействия с базами данных

0.5.1 Определение и сущность Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG), или генерация, дополненная поиском, представляет собой архитектурный подход в области искусственного интеллекта, который объединяет механизмы поиска информации (Retrieval) с генеративными возможностями больших языковых моделей (LLM). Основная концепция RAG заключается в предоставлении LLM доступа к внешним источникам непараметрических знаний, таким как базы данных или корпоративные хранилища документов, непосредственно перед формированием ответа. [RAG-habr]

В рамках этой парадигмы процесс обработки запроса разделяется на два ключевых этапа:

- а) Поиск (Retrieval): Система RAG извлекает релевантные фрагменты информации (контекст) из внешней базы знаний, используя методы семантического поиска. [энциклопедия]
- б) Генерация (Generation): LLM получает исходный запрос пользователя, дополненный извлеченным контекстом, и на этой основе формирует фактологически точный ответ. [RAG-habr, энциклопедия]

Главная задача RAG - преодоление фундаментальных ограничений LLM, таких как ограниченный объем знаний, зафиксированный на момент

обучения, и склонность к "галлюцинациям"(генерации недостоверной информации). Благодаря интеграции актуальных внешних данных RAG значительно повышает фактическую точность и достоверность генерируемых результатов. Кроме того, RAG-системы могут ссылаться на использованные источники, что обеспечивает прозрачность и повышает доверие пользователей. [энциклопедия]

0.5.2 Эволюция и разновидности архитектур RAG

Эволюция RAG отражает стремление к созданию более интеллектуальных и ресурсоэффективных систем за счет оптимизации компонентов поиска и генерации.

Naïve RAG (Наивный RAG) является начальной методологией, основанной на простом последовательном выполнении трех этапов: индексация, поиск и генерация. Документы преобразуются в векторные эмбединги и сохраняются; при запросе ищутся K наиболее релевантных фрагментов по семантическому сходству, которые напрямую передаются LLM. Основной недостаток этой архитектуры — невысокая точность поиска и риск включения нерелевантного контекста, что может приводить к ошибкам генерации.

Advanced RAG (Продвинутый RAG) фокусируется на улучшении качества поиска путем внедрения стратегий оптимизации на этапах до- и после поиска. На этапе pre-retrieval используются такие методы, как оптимизация структуры индекса, уточнение гранулярности данных или переписывание пользовательского запроса (query rewriting) для повышения точности. На этапе post-retrieval применяются техники переранжирования (rerank) и сжатия контекста (context compressing), которые помогают выделить наиболее значимую информацию для LLM.

Modular RAG (Модульный RAG) представляет собой гибкую архитектуру, которая позволяет заменять и реконфигурировать компоненты, отходя от строго последовательного процесса. В эту парадигму вводятся специализированные модули, такие как модуль маршрутизации (Routing), который определяет оптимальный путь обработки запроса, или модуль памяти (Memory) для поддержки многошаговых диалогов.

Особое значение в контексте Text-to-SQL приобретает GraphRAG, который предлагает структурированный и иерархический подход к RAG.

Вместо работы с плоскими текстовыми фрагментами GraphRAG извлекает из исходных данных графы знаний (knowledge graphs), организует схему в иерархическую структуру и использует эти отношения для более точного связывания схемы и генерации SQL-запросов. [Welcome graphing] [NL to SQL]

0.5.3 Проблема интеграции полного контекста базы данных в Text-to-SQL

Для успешной трансляции запроса на естественном языке (NL) в запрос SQL (Text-to-SQL) LLM должна понимать структуру целевой реляционной базы данных (РБД). [RAG размер и качество] Включение полного контекста РБД, включая полную схему отношений (DDL), описания всех таблиц и столбцов, а также, возможно, примеры данных, в промпт LLM не представляется возможным или оправданным из-за ряда технических и экономических ограничений.

3.1. Ограничения контекстного окна и проблема длины промпта

Одним из ключевых барьеров является лимит контекстного окна (context window) LLM, который определяет максимальное количество токенов, обрабатываемых в одном запросе. В случае сложных корпоративных баз данных, содержащих сотни таблиц с комплексными взаимосвязями, полное DDL-описание и сопутствующие метаданные могут легко превысить этот лимит. [rag-habr]

Даже при наличии расширенных контекстных окон (до 100k+ токенов) передача избыточного объема информации снижает производительность LLM. Если промпт содержит много нерелевантных данных, LLM может "потеряться в середине" (Lost in the middle), что приводит к игнорированию критически важной информации для генерации SQL-запроса, несмотря на её физическое присутствие в промпте. [rag review]

3.2. Экономическая неэффективность и стоимость

Эксплуатационные расходы на использование LLM (особенно проприетарных моделей через API) напрямую коррелируют с количеством обработанных токенов. Передача полного контекста РБД, который может быть объемным, в каждом запросе Text-to-SQL приводит к значительному увеличению потребления токенов и, соответственно, к высокой экономической неэффективности решения. [Text to SQL in era llm, rag-habr] Поскольку LLM приходится обрабатывать большой объем избыточного

”шума”в каждом цикле генерации SQL-запроса, стоимость решения становится неоправданно высокой по сравнению с его фактической ценностью. [Text to SQL in era llm, rag-habr]

3.3. Безопасность данных и конфиденциальность

В задачах Text-to-SQL, особенно при работе с корпоративной или конфиденциальной информацией, существует высокий риск нарушения безопасности данных. Если система использует внешние API, отправка полного DDL-описания, а тем более примеров данных (cell values), может привести к нежелательной утечке конфиденциальной информации на внешние серверы [rag размер и качество]. Для соответствия регуляторным требованиям (например, GDPR) и обеспечения конфиденциальности, необходимо минимизировать объем передаваемой информации и гарантировать, что LLM работает только с минимальным, строго необходимым подмножеством метаданных.

0.5.4 Механизм RAG для извлечения релевантного контекста схемы БД

Для преодоления проблемы перегрузки контекста RAG-системы в Text-to-SQL используют механизм извлечения только релевантного подмножества схемы БД, который обеспечивает эффективное связывание естественного языка со схемой (Schema Linking). [NL to SQL] [Text to SQL in era llm]

Индексация структурных метаданных

На этапе офлайн-индексации метаданные базы данных, включая DDL-описания таблиц, имена столбцов, описания их атрибутов, взаимосвязи (внешние ключи) и, опционально, примеры данных, фрагментируются и преобразуются в векторные эмбединги. [rag-habr] В отличие от традиционного RAG, где индексируются полные документы, здесь индексируются структурные элементы БД. Эти эмбединги, отражающие семантическое содержание каждого элемента, сохраняются в векторной базе данных (Vector DB), оптимизированной для поиска ближайших соседей.

Семантический поиск релевантных элементов

При поступлении NL-запроса пользователя, он также кодируется в векторное представление. Затем механизм поиска выполняет семантический поиск в векторной базе данных, используя метрики близости, такие как косинусное сходство. Целью этого этапа является извлечение минимального,

но достаточного набора элементов схемы, которые релевантны запросу.

Ключевые извлекаемые элементы:

- **Релевантные таблицы и столбцы:** Система идентифицирует, какие таблицы и столбцы из тысяч доступных наиболее точно соответствуют сущностям и атрибутам, упомянутым в NL-запросе. [Text to SQL in era llm]
- **Метаданные:** Извлекаются описания столбцов, а также примеры значений, что критически важно для генерации точных предикатов в WHERE-клаузах. [Text to SQL in era llm]
- **Связи:** Извлекается информация о связях между релевантными таблицами, что позволяет LLM корректно формулировать операторы JOIN. [Text to SQL in era llm]

Таким образом, RAG обеспечивает LLM сфокусированным контекстом, минимизируя шум, снижая потребление токенов и обеспечивая надежную основу для генерации SQL.

Генерация SQL-запроса

На финальном этапе LLM-генератор получает структурированный промпт, включающий исходный NL-запрос, извлеченное RAG-системой подмножество схемы БД и специфические инструкции [rag размер и качество]. Основываясь на этом точном контексте, LLM выполняет задачу Text-to-SQL, генерируя синтаксически корректный SQL-запрос, который максимально соответствует семантике пользовательского намерения [rag размер и качество, nl to SQL].

0.6 Методология комплексной оценки систем Text-to-SQL

Детальный анализ качества генерации SQL-запросов в архитектурах, использующих RAG, требует многоуровневого подхода. В отличие от классических задач генерации текста, здесь критически важна синтаксическая валидность, семантическая эквивалентность и вычислительная эффективность.

0.6.1 Формальные метрики соответствия кода

Формальные метрики соответствия кода

Метод ESM

Метод ESM направлен на проверку структурного соответствия между сгенерированным запросом Q_{pred} и эталонным Q_{gold} . Данный подход нивелирует различия в написании, не влияющие на логику (например, порядок условий в предложении WHERE).

- **Процесс декомпозиции:** Запрос парсится в абстрактное синтаксическое дерево (AST) или стандартизированный формат (например, библиотекой sqlglot или внутренними парсерами бенчмарка Spider).
- **Компонентный анализ:** Оценка производится по отдельным блокам: SELECT, WHERE, GROUP BY, ORDER BY, FROM. Для каждого блока вычисляется точность (Precision) и полнота (Recall).
- **F1-Score компонентов:** Итоговый балл ESM часто дополняется оценкой частичного совпадения (Partial Matching), что позволяет дифференцировать модели, совершившие незначительную ошибку в имени столбца, от моделей, полностью неверно определивших структуру запроса.

Execution Accuracy (EX)

EX является наиболее строгой метрикой, так как она подтверждает функциональную идентичность запросов. Результат выполнения $R_{pred} = execute(Q_{pred}, D)$ сравнивается с $R_{gold} = execute(Q_{gold}, D)$

- Проблематика «пустых множеств»: Если оба запроса возвращают пустой результат из-за отсутствия данных в тестовой БД, EX может быть ложноположительной. Для решения этой проблемы в современных тестах (например, BIRD) используются «интенсивные» базы данных с гарантированным наличием записей под каждый тип условий.

–

Учет порядка (Ordering): Если в эталонном запросе отсутствует ORDER BY, при сравнении результатов R_{pred} и R_{gold} наборы строк рассматриваются как неупорядоченные множества.

0.6.2 Продвинутое метрики и бенчмарки (BIRD и Spider)

Для магистерского исследования принципиально различать подходы двух доминирующих бенчмарков.

Spider (Cross-domain & Complex):

- Вводит классификацию запросов по 4 уровням сложности на основе количества джойнов (JOIN), вложенных запросов и агрегатных функций.
- Основной упор делается на Component Matching, что позволяет исследователю понять, на каких именно конструкциях (например, HAVING) модель теряет точность.

BIRD-SQL (Big Bench for Large-scale Database):

- Valid Efficiency Score (VES): Инновационная метрика, которая учитывает не только корректность результата, но и время выполнения запроса. Это критично для RAG-систем, работающих с Big Data, где неоптимальный JOIN может привести к таймауту.
- External Knowledge: BIRD проверяет способность модели использовать внешние словари или описание предметной области, передаваемые в контексте (prompt), что напрямую коррелирует с эффективностью RAG.

Оценка эффективности Retrieval в контуре RAG

В архитектурах Retrieval-Augmented Generation для задач Text-to-SQL качество итоговой генерации детерминировано релевантностью извлеченных метаданных: схем баз данных, описаний атрибутов и примеров кортежей. В связи с этим необходима изолированная оценка компонента извлечения (Retrieval) с использованием следующих метрик:

- **Schema Linking Precision/Recall (P_{sl}, R_{sl}):** Отражает точность и полноту извлечения таблиц и столбцов, необходимых для построения эталонного запроса Q_{gold} . Математически полнота извлечения (Recall) определяется как:

$$R_{sl} = \frac{|S_{pred} \cap S_{gold}|}{|S_{gold}|}$$

где S_{pred} — множество извлеченных элементов схемы, S_{gold} — множество элементов, присутствующих в эталонном запросе. Значение $R_{sl} < 1.0$ свидетельствует о том, что генератор (LLM) априори лишен информации, необходимой для корректной дедукции SQL-кода.

- **Context Relevancy (CR):** Отношение количества релевантных элементов схемы в сформированном промпте к общему объему переданного контекста. Высокая доля нерелевантной информации

(шума) в контексте коррелирует с ростом вероятности «галлюцинаций», при которых модель оперирует семантически близкими, но технически неверными идентификаторами объектов БД.

Интегральная оценка (Faithfulness и SQL-валидность)

Для комплексного анализа системы предлагается использовать интегральный коэффициент, учитывающий не только совпадение с эталоном, но и формальные свойства кода:

- а) **Syntactic Validity** (V_{syn}): Доля запросов, успешно прошедших синтаксический анализ (parsing) и не содержащих ошибок компиляции.
- б) **Semantic Alignment**: Качественная оценка соответствия логики сгенерированного запроса Q_{pred} интену пользователя. Реализуется через парадигму *LLM-as-a-judge* (например, с применением модели GPT-4o), где проверяется соответствие предикатов и агрегатных функций исходной бизнес-логике вопроса.

Таблица 3 – Сравнительный анализ методов оценки Text-to-SQL

Метрика	Объект оценки	Преимущества	Недостатки
ESM	Структура кода	Высокая скорость, интерпретируемость	Игнорирует альтернативные синтаксические конструкции
EX	Результат (data)	Максимальная объективность	Риск ложноположительных срабатываний на пустых БД
VES	Производительность	Устойчивость к вариациям, высокая вычислительная эффективность	Чувствительность к аппаратному обеспечению
Recall@K	Retrieval-контекст	Локализация ошибок на этапе поиска	Не гарантирует валидность финального SQL-кода

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Иванов И. И. Разработка интеллектуальных агентов для баз данных // Вестник информационных технологий. — 2023. — Т. 28, № 3. — С. 45—53.
2. LangChain Documentation. — 2024. — URL: <https://python.langchain.com> ; Дата обращения: 06.11.2025.
3. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows / F. Lei [и др.] // arXiv preprint arXiv:2411.07763. — 2024.
4. Cheng J., Reddy S., Saraswat V. Learning an Executable Neural Semantic Parser // Computational Linguistics. — 2019. — Т. 45, № 1. — С. 59—94.
5. Few-shot Text-to-SQL Translation using Structure and Content Prompt Learning / Z. Gu [и др.] // arXiv preprint arXiv:2301.02111. — 2023.
6. Li F., Jagadish H. V. Understanding Natural Language Queries over Relational Databases // SIGMOD Record. — 2016. — Т. 45, № 2. — С. 6—13.
7. Exploring Dimensions of Generalizability and Few-shot Adaptation in Text-to-SQL Semantic Parsing / R. Patil [и др.] // arXiv preprint arXiv:2305.15637. — 2023.
8. Bazaga A., Scerri K., Abela C. Translating synthetic natural language to database queries // Scientific Reports. — 2021. — Т. 11. — С. 1—16.
9. Multitask Pretraining with Structured Knowledge for Text-to-SQL Generation / R. Giaquinto [и др.] // arXiv preprint arXiv:2301.01933. — 2023.
10. ACT-SQL: In-Context Learning for Text-to-SQL with Automatically-Generated Chain-of-Thought / H. Zhang [и др.] // arXiv preprint arXiv:2310.02901. — 2023.
11. X-SQL: Expert Schema Linking and Understanding of Text-to-SQL with Multi-LLMs / D. Peng [и др.] // arXiv preprint arXiv:2405.20015. — 2024.
12. Katsogiannis-Meimarakis M., Xydias M. Natural Language Interfaces for Databases with Deep Learning: A Survey // VLDB Journal. — 2023. — Т. 32, № 5. — С. 3878—3919.
13. Schema Linking and Understanding: Advances in Text-to-SQL Systems / T. S. Murthy [и др.] // arXiv preprint arXiv:2504.08753. — 2025.

14. Chaturvedi S., Chadha A., Bindschaedler L. SQL-of-Thought: Multi-agentic Text-to-SQL with Guided Error Correction // arXiv preprint arXiv:2509.00581. — 2025.
15. Text-to-SQL Generation using Large Language Model for EHR Analysis / S. B. Joy [и др.] // arXiv preprint arXiv:2406.10786. — 2024.
16. Inc. D. RAG (Retrieval Augmented Generation) on Databricks. — 2024. — URL: <https://docs.databricks.com/aws/en/generative-ai/retrieval-augmented-generation> ; Дата обращения: 06.11.2025.
17. Various. From Natural Language to SQL: Review of LLM-based Text-to-SQL with RAG // arXiv preprint arXiv:2410.01066. — 2024.
18. Fotso S. T. Natural Language Query Engine for Relational Databases // arXiv preprint arXiv:2410.07144. — 2024.
19. BIS: NL2SQL Service Evaluation Benchmark for Business Intelligence / I. Kravets [и др.] // arXiv preprint arXiv:2410.22925. — 2024.
20. Ensuring Data Accuracy in Text-to-SQL Systems / P. Pandey [и др.] // International Journal of Computer Trends and Technology. — 2024. — Т. 72, № 12. — С. 17—24.
21. Rivas P. BERT Goes to SQL School: Improving Automatic Grading of SQL Queries. — 2023. — URL: <https://www.rivas.ai/pdfs/rivas2023bert.pdf>.
22. On Deep Learning Approaches for Single-Turn Text-to-SQL / T. Murthy [и др.] // Texila Journal. — 2024.
23. Inc. L. LangChain: The platform for reliable agents. — 2024. — URL: <https://github.com/langchain-ai/langchain> ; Дата обращения: 06.11.2025.
24. Inc. S. Streamlit Documentation - Create data apps in minutes. — 2024. — URL: <https://streamlit.io> ; Дата обращения: 06.11.2025.
25. Developers S. FastAPI: Modern, fast (high-performance) web framework for building APIs with Python. — 2024. — URL: <https://fastapi.tiangolo.com> ; Дата обращения: 06.11.2025.
26. Group P. G. D. PostgreSQL: The World's Most Advanced Open Source Database. — 2024. — URL: <https://www.postgresql.org/docs/current/index.html> ; Дата обращения: 06.11.2025.

27. Contributors S. SQLAlchemy: The Database Toolkit for Python. — 2024. — URL: <https://github.com/sqlalchemy/sqlalchemy> ; Дата обращения: 06.11.2025.
28. Contributors O. Ollama - Get up and running with Llama 2, Mistral, Dolphin and other models. — 2024. — URL: <https://ollama.com> ; Дата обращения: 06.11.2025.
29. Inc. P. Plotly: Open-source data visualization library. — 2024. — URL: <https://plotly.com> ; Дата обращения: 06.11.2025.
30. OpenAI. OpenAI API Models Documentation. — 2024. — URL: <https://platform.openai.com/docs/models> ; Дата обращения: 06.11.2025.
31. Inc. H. F. Transformers: State-of-the-art Machine Learning for PyTorch, TensorFlow, and JAX. — 2024. — URL: <https://huggingface.co/transformers/> ; Дата обращения: 06.11.2025.
32. Developers P. Pydantic: Data validation using Python type hints. — 2024. — URL: <https://docs.pydantic.dev/latest/> ; Дата обращения: 06.11.2025.
33. Text-to-SQL: A Developer's Zero-to-Hero Guide / I. Shraer [и др.] // TigerData Blog. — 2024. — URL: <https://www.tigerdata.com/learn/text-to-sql-a-developers-zero-to-hero-guide>.
34. Bsat R. Prompt Engineering for a Better SQL Code Generation With LLMs // DataMinded Blog. — 2024. — URL: <https://www.dataminded.com/resources/prompt-engineering-for-a-better-sql-code-generation-with-llms>.
35. Community D. O. LangChain: A Beginner's Guide to Harness the Power of Large Language Models // Digital Ocean Tutorials. — 2024. — URL: <https://www.digitalocean.com/community/tutorials/langchain-language-model>.
36. Team A. Use Natural Language to Generate SQL Queries // AskYourDatabase Blog. — 2024. — URL: <https://www.askyourdatabase.com/blog/use-natural-language-to-generate-sql-queries>.
37. Team W. How to Use LLM with RAG to Chat with Databases: Complete Guide to SQL Query Generation with Natural Language // Webasha Blog. — 2025. — URL: <https://www.webasha.com/blog/how-to-use-llm-with-rag-to-chat-with-databases>.

38. Team G. Implementing Semantic Search with Vector Database // GeeksforGeeks. — 2025. — URL: <https://www.geeksforgeeks.org/data-science/implementing-semantic-search-with-vector-database/>.
39. Milvus. How do Vector Embeddings Work in Semantic Search? // Milvus Blog. — 2024. — URL: <https://milvus.io/ai-quick-reference/how-do-vector-embeddings-work-in-semantic-search>.
40. Team K. Semantic Search with Vector Databases // KDnuggets. — 2025. — URL: <https://www.kdnuggets.com/semantic-search-with-vector-databases>.
41. Team Z. What is Query Disambiguation in Search Systems? // Zilliz Blog. — 2024. — URL: <https://zilliz.com/ai-faq/what-is-query-disambiguation-in-search-systems>.
42. Services A. W. Optimizing PostgreSQL Query Performance // AWS Prescriptive Guidance. — 2024. — URL: <https://docs.aws.amazon.com/prescriptive-guidance/latest/postgresql-query-tuning/introduction.html>.
43. Team T. Best Practices for Query Optimization on PostgreSQL // TigerData Blog. — 2024. — URL: <https://www.tigerdata.com/blog/best-practices-for-query-optimization-in-postgresql>.
44. Sky J. A. Improving Performance of PostgreSQL Queries // DEV Community. — 2024. — URL: <https://dev.to/jacobandrewsky/improving-performance-of-postgresql-queries-1h7o>.
45. Team T. Real-time Data Visualization: How to Build Faster Dashboards // Tinybird Blog. — 2023. — URL: <https://www.tinybird.com/blog/real-time-data-visualization>.
46. Ramoliya K. HI-SQL: Optimizing Text-to-SQL Systems through Dynamic Prompt Optimization // Keyur's Blog. — 2025. — URL: <https://keyurramoliya.com/posts/HI-SQL/>.
47. Services A. W. Build a Robust Text-to-SQL Solution with Self-Correcting Queries // AWS Machine Learning Blog. — 2024. — URL: <https://aws.amazon.com/blogs/machine-learning/build-a-robust-text-to-sql-solution-generating-complex-queries-self-correcting-and-querying-diverse-data-sources/>.

48. Team F. SQL Agent: Building Intelligent Query Systems with Error Correction // FlowiseAI Documentation. — 2025. — URL: <https://docs.flowiseai.com/tutorials/sql-agent>.
49. Research G. SQLPrompt: Improved In-context Learning for Few-shot Text-to-SQL // Google Research Publications. — 2022. — URL: <https://research.google/pubs/sqlprompt-improved-in-context-learning-for-few-shot-text-to-sql/>.
50. Team C. The Art of Prompt Engineering // CID Blog. — 2025. — URL: <https://cid.com/blog/the-art-of-prompt-engineering/>.
51. Contributors W. Named-entity recognition // Wikipedia. — 2024. — URL: <https://en.wikipedia.org/wiki/Named-entity-recognition>.
52. Microsoft. How to use Named Entity Recognition (NER) // Azure AI Services Documentation. — 2025. — URL: <https://learn.microsoft.com/en-us/azure/ai-services/language-service/named-entity-recognition/how-to-call>.
53. Team G. Named Entity Recognition in NLP // GeeksforGeeks. — 2021. — URL: <https://www.geeksforgeeks.org/nlp/named-entity-recognition/>.
54. Auden. How to Run AI Models Locally with Ollama: Deploy LLMs and Debug APIs in Minutes // DEV Community. — 2025. — URL: <https://dev.to/auden/how-to-run-ai-models-locally-with-ollama-deploy-llms-and-debug-apis-in-minutes-59pc>.
55. Team A. Ollama vs. vLLM: The Definitive Guide to Local LLM Frameworks in 2025 // AlphaBravo Blog. — 2025. — URL: <https://blog.alphabravo.io/ollama-vs-vllm-the-definitive-guide-to-local-llm-frameworks-in-2025/>.
56. Team R. Deploy Local Models with Ollama and Xinference // RAGFlow Documentation. — 2024. — URL: https://ragflow.io/docs/dev/deploy_local_llm.
57. Lear S. Streamlit 101 - Create a Simple Data Visualization App in 20 Minutes // Substack. — 2024. — URL: <https://sarahleaschrch.substack.com/p/streamlit-101-create-a-simple-data>.
58. Community P. H. Creating a Dashboard for Interactive Data Visualization // Programming Historian. — 2025. — URL: <https://programminghistorian.org/en/lessons/interactive-data-visualization-dashboard>.

59. Team P. A. Building Microservices with FastAPI: A Comprehensive Guide // Prama AI Blog. — 2024. — URL: <https://prama.ai/building-microservices-with-fastapi-a-comprehensive-guide/>.
60. Team U. Streamlit vs Dash: Which Python framework is best for you? // UIBakery Blog. — 2025. — URL: <https://uibakery.io/blog/streamlit-vs-dash>.
61. Team B. Top 10 Python REST API Frameworks in 2024 // BrowserStack Blog. — 2024. — URL: <https://www.browserstack.com/guide/top-python-rest-api-frameworks>.
62. Platform E. C. Querying Relational Databases With SQLAlchemy in Python // Earthly Blog. — 2023. — URL: <https://earthly.dev/blog/query-database-sqlalchemy-python/>.
63. Team D. Ultimate guide to SQLAlchemy library in Python // Deepnote Blog. — 2024. — URL: <https://deepnote.com/blog/ultimate-guide-to-sqlalchemy-library-in-python>.
64. Contributors W. PostgreSQL // Wikipedia. — 2024. — URL: <https://en.wikipedia.org/wiki/PostgreSQL>.
65. Team L. B. 7 Best Chart Libraries For Developers in 2024 // Latitude Blog. — 2024. — URL: <https://latitude-blog.ghost.io/blog/7-best-chart-libraries-developers-2024/>.
66. Team T. Short guide: Plotting data in Python with Plotly (2024) // TensorScience Blog. — 2023. — URL: <https://www.tensorscience.com/guides/short-guide-plotting-data-in-python-with-plotly-2024>.
67. Team D. Python Visualization in 2024 // Deepnote Blog. — 2023. — URL: <https://deepnote.com/blog/python-visualization-in-2024>.
68. Team T. 7 Best Python Visualization Libraries for 2024 // DEV Community. — 2024. — URL: <https://dev.to/taipy/7-best-python-visualization-libraries-for-2024-5h9f>.
69. Team S. Data Visualization with Streamlit, Dash, and Panel. Part 1 // Sunscrapers Blog. — 2023. — URL: <https://sunscrapers.com/blog/data-viz-streamlit-dash-panel-part-1/>.

70. Team C. Effortless Declarative Data Validation with Pydantic // CoDiLime Blog. — 2025. — URL: <https://codilime.com/blog/declarative-data-validation-pydantic/>.
71. Team P. What is Pydantic? Validating Data in Python // Prefect Blog. — 2024. — URL: <https://www.prefect.io/blog/what-is-pydantic-validating-data-in-python>.
72. SMJ S. Mastering the Hugging Face Transformers Library // Stephen SMJ Blog. — 2024. — URL: <https://stephen-smj.tech/2024/05/25/Transformers/>.
73. Ameenza A. Hugging Face Transformers in 2024: Democratizing AI Development // Anshal's Blog. — 2024. — URL: <https://anshadameenza.com/blog/technology/huggingface-transformers-2024/>.
74. Databricks. What are Hugging Face Transformers? // Databricks Machine Learning Documentation. — 2024. — URL: <https://docs.databricks.com/aws/en/machine-learning/train-model/huggingface/>.
75. Team C. 10 Must-Try Text to SQL Tools for 2024 // Chat2DB Blog. — 2024. — URL: <https://articles.chat2db.ai/10-must-try-text-to-sql-tools-2024/>.
76. Team T. Which LLM writes the best analytical SQL? // Tinybird Blog. — 2025. — URL: <https://www.tinybird.com/blog/which-llm-writes-the-best-sql>.
77. Team C. The Most Popular Python Frameworks in 2024 // Codeanywhere Blog. — 2024. — URL: <https://codeanywhere.com/blog/the-most-popular-python-frameworks-in-2024>.
78. Foundation P. S. Python Programming Language. — 2024. — URL: <https://www.python.org> ; Дата обращения: 06.11.2025.
79. Developers N. NumPy Documentation. — 2024. — URL: <https://numpy.org/doc/> ; Дата обращения: 06.11.2025.
80. Team P. D. Pandas Documentation. — 2024. — URL: <https://pandas.pydata.org/docs/> ; Дата обращения: 06.11.2025.
81. Team F. FastAPI Documentation. — 2024. — URL: <https://fastapi.tiangolo.com/> ; Дата обращения: 06.11.2025.
82. Inc. S. Streamlit Documentation. — 2024. — URL: <https://docs.streamlit.io> ; Дата обращения: 06.11.2025.

83. Plotly. Dash Documentation. — 2024. — URL: <https://dash.plotly.com> ; Дата обращения: 06.11.2025.
84. Group P. G. D. PostgreSQL 18 Documentation. — 2024. — URL: <https://www.postgresql.org/docs/current/index.html> ; Дата обращения: 06.11.2025.
85. Contributors S. SQLAlchemy Documentation. — 2024. — URL: <https://docs.sqlalchemy.org/> ; Дата обращения: 06.11.2025.
86. Contributors S. SQLAlchemy: The Database Toolkit for Python. — 2024. — URL: <https://github.com/sqlalchemy/sqlalchemy> ; Дата обращения: 06.11.2025.
87. Inc. P. Plotly Documentation. — 2024. — URL: <https://plotly.com/python/> ; Дата обращения: 06.11.2025.
88. Team M. D. Matplotlib Documentation. — 2024. — URL: <https://matplotlib.org/> ; Дата обращения: 06.11.2025.
89. Developers S. Seaborn Documentation. — 2024. — URL: <https://seaborn.pydata.org/> ; Дата обращения: 06.11.2025.
90. Inc. L. LangChain Documentation. — 2024. — URL: <https://python.langchain.com> ; Дата обращения: 06.11.2025.
91. Inc. L. LangChain: The platform for reliable agents. — 2024. — URL: <https://github.com/langchain-ai/langchain> ; Дата обращения: 06.11.2025.
92. Inc. H. F. Transformers: State-of-the-art Machine Learning for PyTorch, TensorFlow, and JAX. — 2024. — URL: <https://huggingface.co/transformers/> ; Дата обращения: 06.11.2025.
93. Inc. H. F. Hugging Face Model Hub. — 2024. — URL: <https://huggingface.co/models> ; Дата обращения: 06.11.2025.
94. Contributors O. Ollama. — 2024. — URL: <https://ollama.com> ; Дата обращения: 06.11.2025.
95. Contributors O. Ollama GitHub Repository. — 2024. — URL: <https://github.com/ollama/ollama> ; Дата обращения: 06.11.2025.
96. OpenAI. OpenAI API Documentation. — 2024. — URL: <https://platform.openai.com/docs> ; Дата обращения: 06.11.2025.
97. OpenAI. OpenAI Models. — 2024. — URL: <https://platform.openai.com/docs/models> ; Дата обращения: 06.11.2025.

98. Anthropic. Anthropic Claude API. — 2024. — URL: <https://www.anthropic.com> ; Дата обращения: 06.11.2025.
99. Developers P. Pydantic Documentation. — 2024. — URL: <https://docs.pydantic.dev/latest/> ; Дата обращения: 06.11.2025.
100. Contributors M. Milvus Vector Database Documentation. — 2024. — URL: <https://milvus.io> ; Дата обращения: 06.11.2025.
101. Weaviate. Weaviate Vector Database Documentation. — 2024. — URL: <https://weaviate.io> ; Дата обращения: 06.11.2025.
102. Pinecone. Pinecone Vector Database. — 2024. — URL: <https://www.pinecone.io> ; Дата обращения: 06.11.2025.
103. Chroma. Chroma Vector Database. — 2024. — URL: <https://www.trychroma.com> ; Дата обращения: 06.11.2025.
104. Contributors S. T. Sentence Transformers Documentation. — 2024. — URL: <https://www.sbert.net> ; Дата обращения: 06.11.2025.
105. Lab Y. L. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. — 2024. — URL: <https://yale-lily.github.io/spider> ; Дата обращения: 06.11.2025.
106. Researchers V. BIRD: A Benchmark for Large-Scale Database Grounded Text-to-SQL. — 2024. — URL: <https://bird-bench.github.io> ; Дата обращения: 06.11.2025.
107. Development Team pytest. pytest Documentation. — 2024. — URL: <https://docs.pytest.org> ; Дата обращения: 06.11.2025.
108. Foundation P. S. unittest - Unit testing framework. — 2024. — URL: <https://docs.python.org/3/library/unittest.html> ; Дата обращения: 06.11.2025.
109. Foundation P. S. logging — Logging facility for Python. — 2024. — URL: <https://docs.python.org/3/library/logging.html> ; Дата обращения: 06.11.2025.
110. Authority P. P. pip Documentation. — 2024. — URL: <https://pip.pypa.io/> ; Дата обращения: 06.11.2025.
111. Contributors P. Poetry Documentation. — 2024. — URL: <https://python-poetry.org/> ; Дата обращения: 06.11.2025.

112. Inc. L. LangChain: Retrieval-Augmented Generation (RAG). — 2024. — URL: https://python.langchain.com/docs/use_cases/question_answering/ ; Дата обращения: 06.11.2025.

113. Team L. LlamaIndex (formerly GPT Index). — 2024. — URL: <https://www.llamaindex.ai> ; Дата обращения: 06.11.2025.

114. DeepSet. Haystack Documentation. — 2024. — URL: <https://docs.haystack.deepset.ai> ; Дата обращения: 06.11.2025.

115. Inc. G. GitHub Actions Documentation. — 2024. — URL: <https://docs.github.com/en/actions> ; Дата обращения: 06.11.2025.

116. Inc. D. Docker Documentation. — 2024. — URL: <https://docs.docker.com> ; Дата обращения: 06.11.2025.

117. Authors P. Prometheus Monitoring System. — 2024. — URL: <https://prometheus.io/docs> ; Дата обращения: 06.11.2025.

118. Labs G. Grafana Documentation. — 2024. — URL: <https://grafana.com/docs/> ; Дата обращения: 06.11.2025.